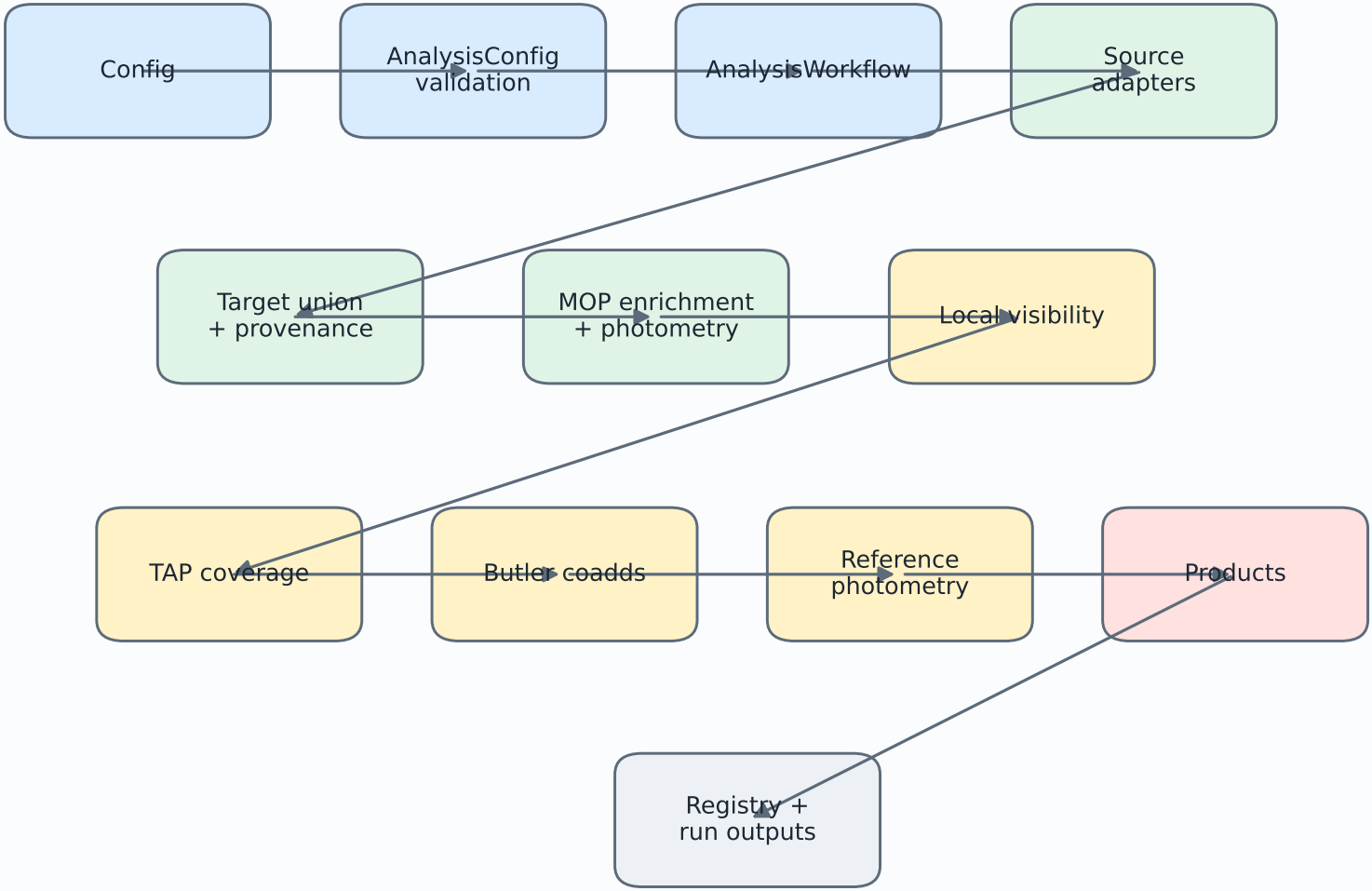
# Target Selection Architecture Proposal

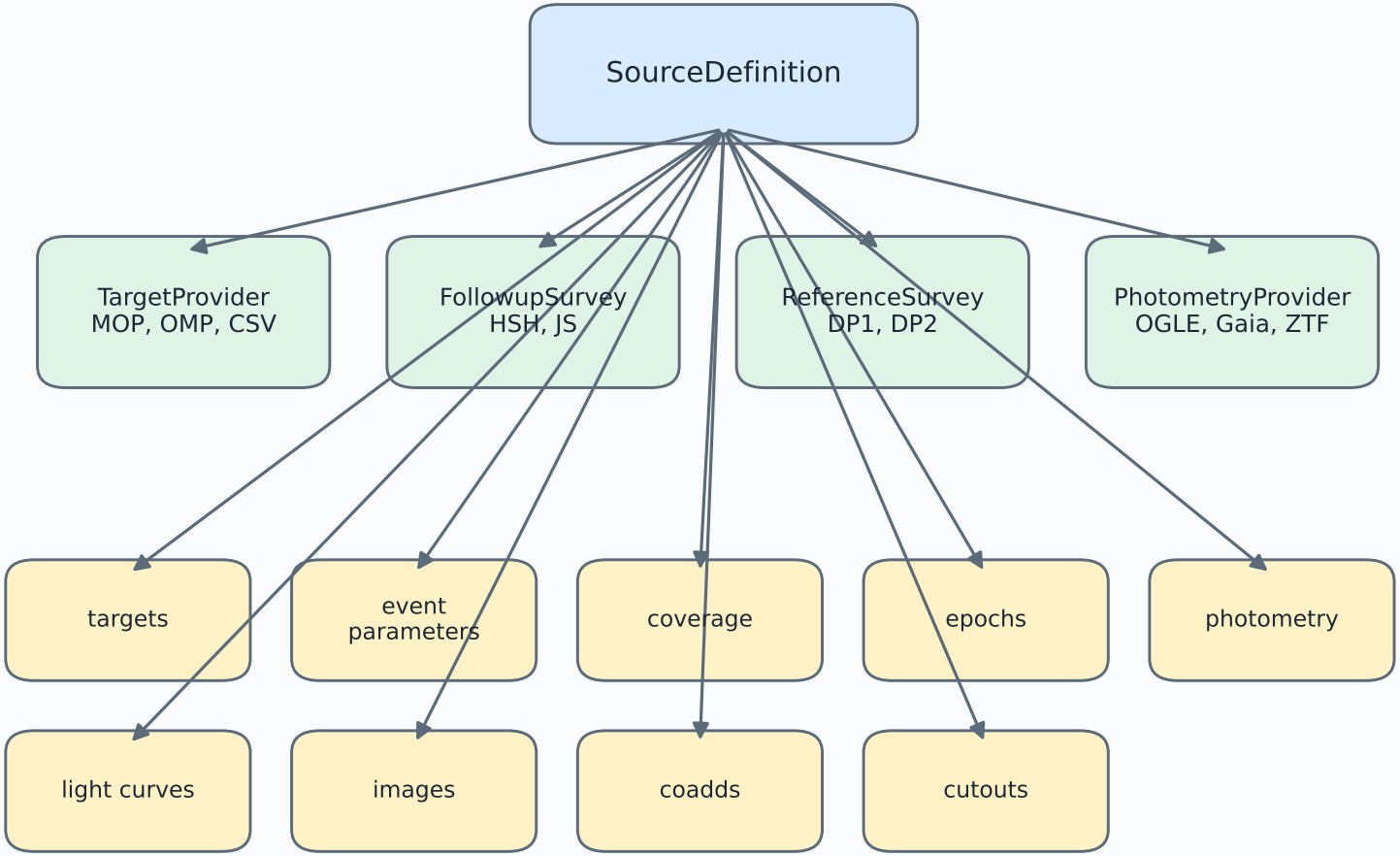
This PDF describes the current organization of target_selection and proposes an incremental architecture that is easier to extend to MOP, OMP, HSH, JS, Rubin DP1/DP2, OGLE, Gaia, ZTF, and future sources. The central recommendation is to keep semantic source roles, add explicit data capabilities, and split the monolithic scientific backend into composable tasks.

# Current Workflow



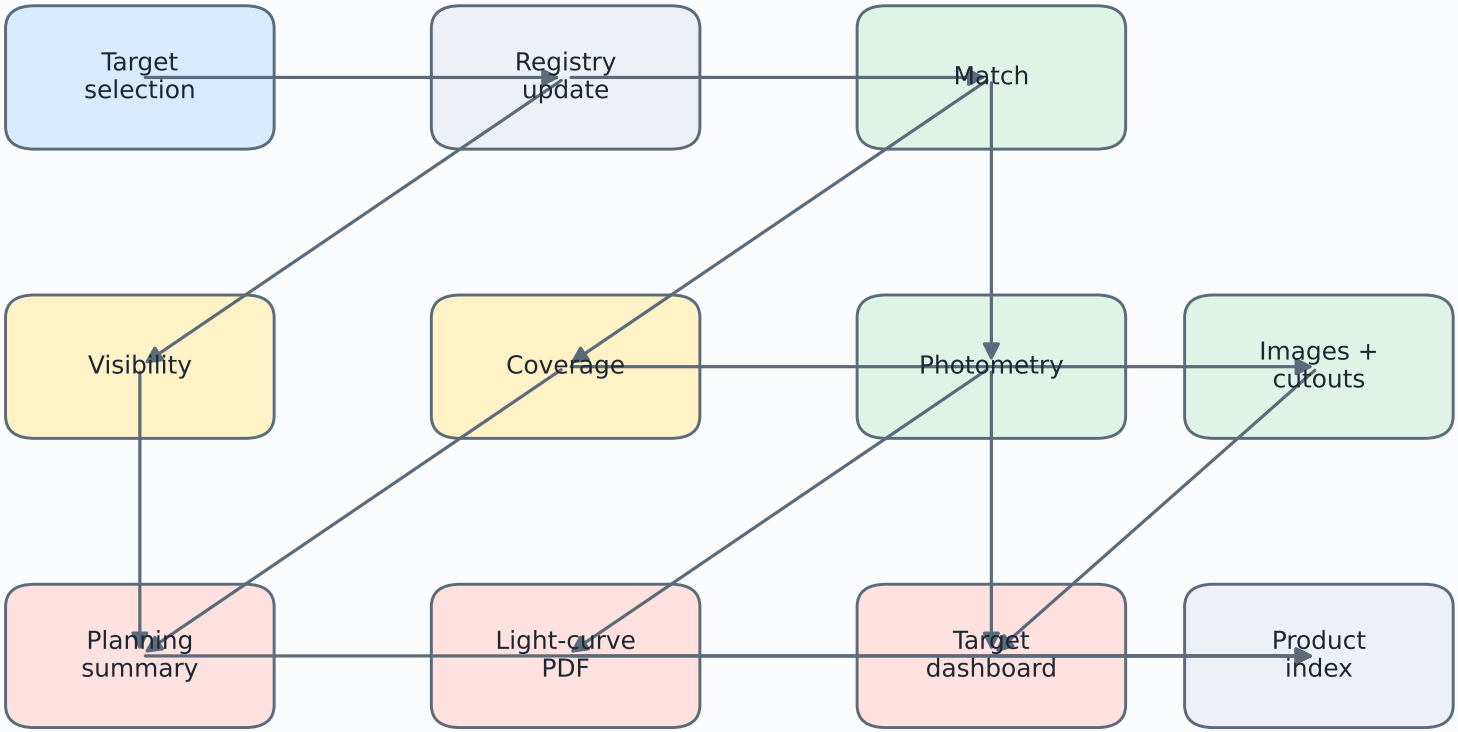

Main current issue: the high-level source/adapters model is good, but target_selection_pipeline.py still owns too many unrelated tasks: visibility, Rubin coverage, coadds, photometry, summaries, sky plots, and dashboards.

# Proposed Source Model



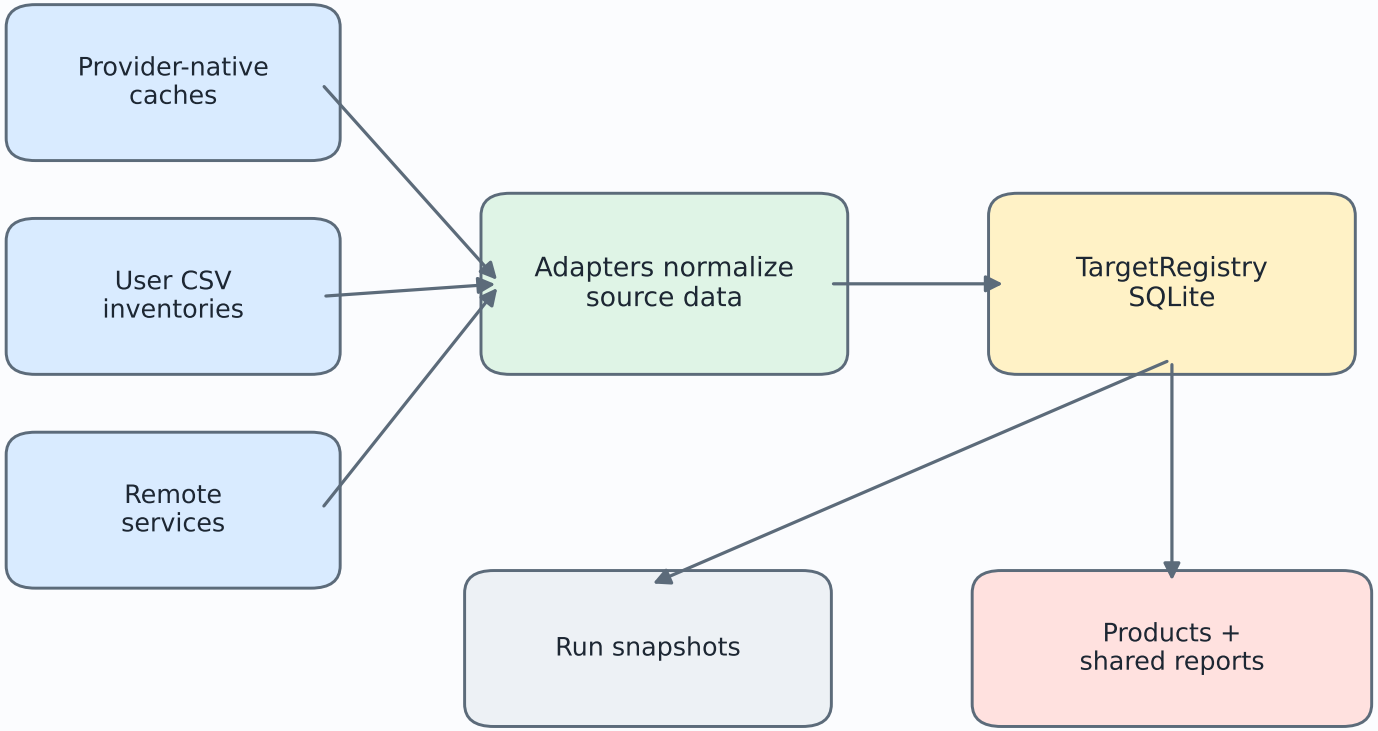

Roles explain why a source participates. Capabilities explain what data it can provide. A source can have several capabilities, and tasks request capabilities instead of hard-coding provider names.

# Proposed Task Model




Each task should declare required inputs, optional inputs, outputs, and compatible source capabilities. This keeps dashboard-only, visibility-only, and photometry-only runs independent.

# Data Ownership




Normalized persistent facts belong in the registry: identities, authoritative coordinates, source records, epochs, photometry, matches, coverage summaries, and run metadata. Large provider-native files and images can remain as files with registry metadata.

# Configuration and Functional Tags

Recommended config direction:

1. Keep semantic source lists: target_providers, followup_surveys, reference_surveys, and optionally photometry_providers.
2. Add source capabilities so tasks can request coverage, photometry, images, coadds, cutouts, or light curves without provider-specific branching.
3. Keep selection settings independent: target_names, bands, magnitude cuts, visibility dates/windows, and data-availability requirements.
4. Make tasks explicit: match, coverage, photometry, images, visibility, planning_summary, monitoring_report, target_dashboard.
5. Keep product switches as the user-facing output layer, but allow tags such as product:target-report or photometry:dia to map to task groups.

Useful tags:

targets:mop-visible, targets:followup-observed, targets:user-list, targets:subset, match:catalog, visibility:local, coverage:survey, photometry:mop, photometry:dia, photometry:coadd-forced, images:coadd, images:cutout, product:dashboard, product:lightcurves, product:planning-table, product:visibility-plots.

# Refactor Recommendation

Recommended refactor path:

1. Preserve run_analysis(config), the CLI, and the notebook workflow.
2. Define capability protocols: TargetProvider, EventDataProvider, CoverageProvider, PhotometryProvider, ImageProvider, and CutoutProvider.
3. Split target_selection_pipeline.py into task modules for coverage, photometry, images, dashboards, summaries, and run assembly.
4. Promote MOP enrichment and photometry into provider capabilities, so MOP can be used for enrichment even when its daily visible target list is not selected.
5. Store match results and reference photometry in the registry with source, method, collection, counterpart ID, separation, status, and version.
6. Keep old product keywords as compatibility aliases while adding task/product groups.
7. Keep target reports in a shared target_reports/ folder and update reports by target name.

Do not rewrite everything at once. The current architecture is usable. The next changes should isolate the monolithic Rubin/product backend first, because that is where most complexity is concentrated.